

CStore: A Browser-Based Studio for AI-Generated and LLM-Revised Csound Instrument Design

Li Shi¹ and Richard Boulanger²

¹ Dartmouth College, Hanover, NH, USA

² Berklee College of Music, Boston, MA, USA

Li.Shi.GR@Dartmouth.edu, rboulanger@berklee.edu

Abstract. Most text-to-music and text-to-audio systems generate a finished waveform: something to listen to, but not something a composer can directly edit, version, or reproduce. CStore instead generates the underlying Csound project file, and unlike established Csound authoring environments such as CsoundQt and Cabbage, it is built around a generative agent trained on a large, curated corpus of Csound instruments. This paper presents the CStore Web Console, a browser-based studio built on top of that generator that lets a composer draft, audition, vary, and revise Csound instruments without leaving the keyboard. The console pairs a Next.js interface with a Flask sidecar that shells out to the Csound command-line renderer, and organizes work around a library of runs, single-key commands for drafting and playback, and two ways of producing variations: deterministic, score-derived “children” and LLM-driven variants tiered by how far they may drift from a “mother” run. A separate path lets a user hand a rendered instrument to one of six language-model providers with a plain-English instruction, such as “add a plate reverb and lower the pitch by an octave,” and get back a new, separately versioned *.csd* rather than an overwritten one. We describe the console’s architecture and workflow, and what building a studio around an editable *.csd* implies for how AI-assisted Csound instruments get made and revised.

Keywords: Csound, agentic AI, Csound-trained LLM, editable *.csd*, human-AI co-editing, instrument design

1 Introduction

A generative model that outputs a finished waveform gives a composer something to listen to but not much to work with. CStore was built on the opposite premise: generate the Csound project file itself, an orchestra and score in plain text, so the output is inspectable, editable, and re-renderable under fixed settings rather than a closed audio object [1]. Concretely, CStore generates a *.csd*: a human-readable document containing a complete instrument definition (orchestra) and event schedule (score), the same artifact a Csound programmer would write by hand [7,8,9,10]. Being plain text, it can be revised at the parameter or code level, version-controlled, and re-rendered deterministically under a pinned Csound version, rather than only regenerated from scratch.

Most Csound users already work inside a mature authoring environment, CsoundQt [13] or Cabbage [14]. What neither provides is a generative layer: CStore is built around an agent trained on a large, curated corpus of Csound instruments, and can draft, vary, and revise a *.csd* from a prompt or plain-English instruction. A *.csd* the console produces opens in either IDE like any other Csound project, upstream of them rather than in place of them.

This paper describes the console built around that generative layer: a browser-based studio wrapping CStore in a keyboard-driven interface for drafting, auditioning, branching, and editing instruments. Every run stays a plain-text artifact fit for diffing, but the console adds fast work across many candidates, a visible lineage between an instrument and its variations, and a second, natural-language revision channel, shown end to end in Fig. 1.

2 Background: The CStore Generator

Most current text-to-music and text-to-audio systems, including MusicLM, AudioLDM, Jukebox, MusicGen, and AudioGen, generate a finished waveform or an implicit token representation as their

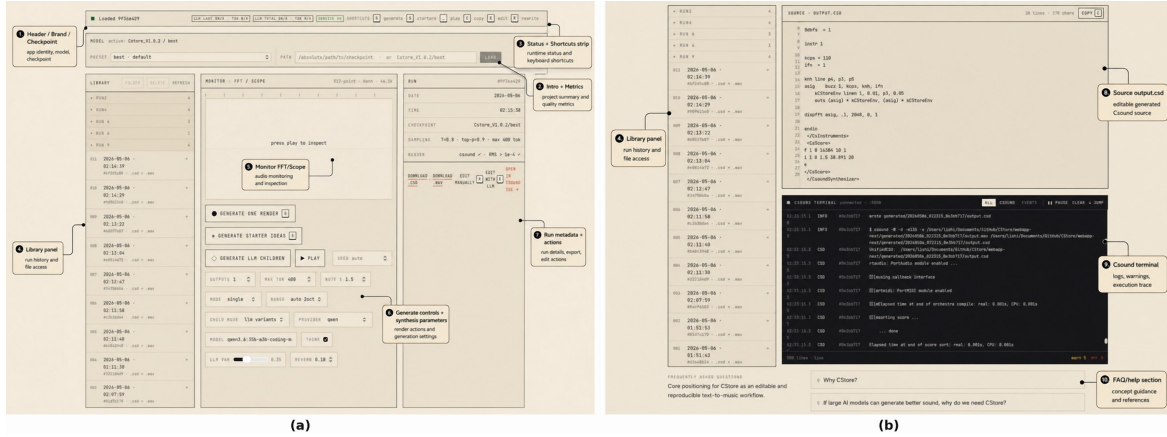


Fig. 1. The CStore Web Console interface. (a) Top half: header with brand and checkpoint controls (1), an intro/metrics summary (2), a status and keyboard-shortcuts strip (3), the library panel of past runs (4), the FFT/scope monitor (5), and the generate controls and synthesis parameters (6), alongside the current run’s metadata and actions (7). (b) Bottom half: the library panel continued (4), the editable generated .csd source (8), the live Csound terminal (9), and the FAQ/help section (10).

primary output [2,3,4,5,6]. CStore differs from these, and from the Csound-native tools of Section 1, by generating an editable Csound specification rather than audio or a black-box token stream.

The generator is an autoregressive, GPT-2-style Transformer (8 layers, 384 hidden dimensions, 8 heads, about 34 million parameters, 512-token context) trained on 27,104 .csd files (17.81 GB) from public Csound archives, instrument catalogs, and community collections [12]. A prompt conditions it as prefix context rather than being parsed into explicit constraints, so CStore is closer to conditioned generation than to guaranteed semantic alignment between request and result. Candidates are rendered offline and scored on three checks, structural completeness, render success, and audible output, following the loop in Fig. 2; full performance figures by training regime appear in the companion paper [1].

To clarify the model’s foundations: CStore’s generator was trained from scratch on the Csound corpus described above, a compact, decoder-only Transformer built to a custom GPT-2-scale configuration rather than adapted from an existing pretrained language model’s weights. It does borrow the standard GPT-2 byte-pair-encoding vocabulary as a starting point, extended with special tokens for Csound’s own structural tags (<CsoundSynthesizer>, <CsInstruments>, <CsScore>, and related markers); the tokenizer grows from 50,257 to 50,265 tokens, with the embedding and output layers resized to match, so each tag becomes one token instead of several fragmented byte-pairs. Prompt conditioning is trained directly, not assumed: a continuation objective cuts a training .csd at a controllable point into a prefix and suffix and masks the loss over the prefix, so the model learns specifically to continue from a given opening, the same mechanism the console’s child-generation and LLM-editing paths rely on at inference time. The baseline model, trained on the full 27,102-document corpus, is then adapted through two further passes: an expert-curated fine-tune on a low-external-dependency subset, and this prefix-conditioned continuation fine-tune. Both are implemented with HuggingFace’s **Trainer** API and released in the public repository [15]; full training hyperparameters, dataset statistics, and layered evaluation metrics (structural validity, render success, audible output) are reported in the companion paper [1].

3 The CStore Web Console

3.1 Architecture and Design Rationale

The console is one project with two processes: a Next.js 16 interface laid out as a studio with a library rail, a transport and source inspector, and per-run editing tools, and a Flask sidecar that loads the PyTorch checkpoint and tokenizer, shells out to the **csound** renderer [11], and writes each run’s .csd, .wav, and a **meta.json** sidecar into a **generated/** directory. The split follows what each half needs:

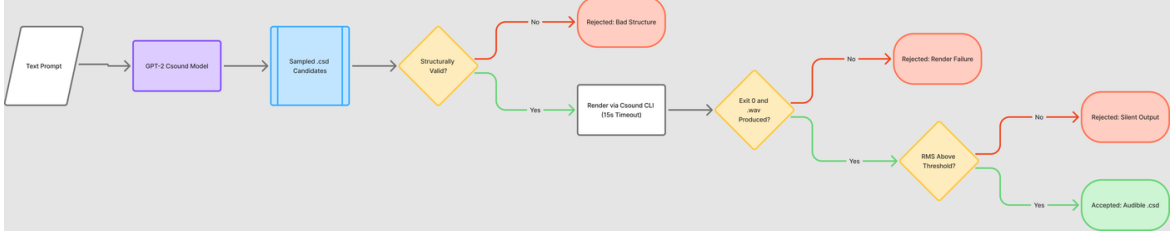


Fig. 2. Generation and evaluation loop: sampled *.csd* candidates are rendered under a fixed Csound CLI and timeout, then labeled for structural validity, render success, and audible output.

the model wants Python, rendering needs the `csound` binary, and neither fits a Node process well. Console and generator are released together in the public CStore repository [15].

The interface is shown in Fig. 1: (a) covers checkpoint loading, run status, the library of past runs, the FFT/scope monitor, and the generation and synthesis controls; (b) covers the library continued, the editable generated source, the live Csound terminal, and the FAQ/help panel. The library and per-run panel carry the mother/child provenance described in Sect. 3.3.

3.2 A Keyboard-Driven Drafting Workflow

Every core action has a single-key shortcut: **G** drafts one render, **S** drafts a batch of starter ideas, **Space** plays or pauses, **C** copies its *.csd*, **E** opens the LLM-editing dialog, and **R** opens a hand-edit window. A library row’s star pins it to Favourites, persisted in the run’s `meta.json`; the rest of the library holds the fifty most recent runs, and older, unfavourited ones roll off. Auditioning generated instruments is closer to flipping through synth patches than writing code: draft, listen, discard or pin, repeat.

3.3 Starters and Children: Building a Family Tree of Variations

The console distinguishes two kinds of new material. A starter is a short, freshly generated idea with no parent; a child is derived from a selected run, and every child card shows its mother run’s identifier (Fig. 1a) so a family tree stays legible after dozens of branches. Starter batches run one to twelve at a time, in three score shapes (single note, arpeggio, chord).

Children come two ways. Classic children are deterministic, score-derived rewrites of the mother, for hearing an instrument across pitch or duration. LLM children instead ask the sound designer for a variation, with a variation-temperature dial in three tiers: low (0.00–0.33), close to the mother; medium (0.34–0.66), a moderate shift in timbre or register; and high (0.67–1.00), an aggressive departure instructed not to resemble the source. This substitutes for finer sampling controls, giving a composer a vocabulary for “close cousin” versus “distant relative.”

3.4 Manual Editing and Deterministic Re-render

Pressing **R** opens a monospace editor seeded from the run’s *.csd*. `Cmd/Ctrl-Enter` sends the edit to the render endpoint, which writes it as a new run and calls Csound directly, no model, no network. If the edit has an error, the run is still saved and the error surfaces in the dialog, so a broken edit stays inspectable.

3.5 Editing with an LLM: Six Providers, One Interface

Pressing **E** sends the active run’s *.csd*, with a plain-English instruction, to one of six providers, and returns a revised *.csd* rendered and saved as a new, separately tracked run; the original is never overwritten. An instruction might read “add a plate reverb, lower the pitch by an octave”; the console passes it through without parsing or verifying it.

The providers span a range of cost and locality: a local option through Ollama (Qwen, no key), a keyless free tier through Pollinations, a free tier through OpenRouter requiring a key, and three paid options, OpenAI, Anthropic, and Gemini. Every edited run records its source run, provider, model,

and instruction. A dollar estimate appears only when per-token rates are configured, otherwise left null. API keys are stored locally at `~/cstore/keys.json` with owner-only file permissions (`chmod 0600`) and are masked whenever the interface displays them. The console is built for a single user running it on their own machine, not as a multi-tenant service, and the local storage reflects that: it is a deliberate design choice, not an oversight. In practice, the paid providers (OpenAI, Anthropic, Gemini) work reliably; the free and keyless options (Pollinations, OpenRouter’s free tier) vary in availability from day to day, so finding a working free provider currently involves some trial and error. Recommending, and periodically re-verifying, which free options are reliable is future work.

4 Discussion and Future Work

The console does not change what CStore generates; it changes what a person does with what it generates. Never overwriting a run turns a session into an append-only history, keeping mother/child lineage legible after many branches. Keeping manual and LLM editing as two labeled paths avoids collapsing human and model changes into one category. The generator is deliberately unforgiving about what counts as a result: a candidate that fails to render, or renders silently, is never surfaced to the user, the system simply keeps sampling against the same prompt until a structurally valid, audible *.csd* appears (Fig. 2; full layered metrics in [1]). From the console’s side, this functions as an adaptive filter: the interface only ever shows instruments the generator has already confirmed work, so drafting with **G** or auditioning a batch with **S** never means listening through or discarding failed renders yourself. The tradeoff is latency and compute rather than user-facing failure, and it is why a single-key workflow suits rapid audition better than the slower, code-first work a programmer might do in CsoundQt or Cabbage, hence the manual-edit path.

Two efforts take priority over the console itself: further curation and training aimed at the generator’s most common failure, not knowing where DSP- and sample-based instruments expect their samples, rather than only filtering for them, and a tokenizer gaining more Csound-specific tokens for a wider sonic vocabulary.

References

1. Shi, L., Boulanger, R.: CStore: A Framework for Generating Editable, Versionable Csound Orchestra Code - From Text to Sound and Music. In: 2026 IEEE Conference on Artificial Intelligence (CAI), Granada, Spain (2026), <https://doi.org/10.1109/CAI68641.2026.11536429>
2. Agostinelli, A. et al.: MusicLM: Generating Music From Text. arXiv:2301.11325 (2023)
3. Liu, H. et al.: AudioLDM: Text-to-Audio Generation with Latent Diffusion Models. arXiv:2301.12503 (2023)
4. Dhariwal, P. et al.: Jukebox: A Generative Model for Music. arXiv:2005.00341 (2020)
5. Copet, J. et al.: Simple and Controllable Music Generation. arXiv:2306.05284 (2023)
6. Kreuk, F. et al.: AudioGen: Textually Guided Audio Generation. arXiv:2209.15352 (2022)
7. Lazzarini, V. et al.: Csound: A Sound and Music Computing System. Springer (2016), <https://link.springer.com/book/10.1007/978-3-319-45370-5>
8. Boulanger, R. (ed.): The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming. MIT Press (2000), <https://mitpress.mit.edu/9780262522618/the-csound-book/>
9. Csound Community: The Canonical Csound Reference Manual, Version 6.18.0 (2022), <https://csound.com/docs/manual/index.html>
10. Csound Community: The Csound FLOSS Manual, <https://flossmanual.csound.com/>
11. Csound. GitHub, <https://github.com/csound/csound>
12. Csound Home Page. <https://csound.com/>
13. CsoundQt. <https://csoundqt.github.io/>
14. Cabbage Audio. <https://cabbageaudio.com/>
15. CStore repository. GitHub, <https://github.com/CharlieSL1/CStore>